

University of Hertfordshire

School of Physics, Engineering and Computer Science

Multiplayer Memory Game Project

Module: 4FTC2029-0105-Computer Programming for Electronics Engineer

Group 03 Team10

Group Members:

1. Youssef Mahgoub

2. Youssef Rafik

3. Mahmoud Gamal

4. Hussein Khorsheid

Submission Date: 28th of April 2026

Table of Contents:-

1. Introduction	1
2. Project Objectives	1
3. System Overview	2
4. Hardware	2
4.1 Component List	2
4.2 Hardware Implementation	3
5. Software and Code	4
5.1 How the Game Software Works	4
5.2 Button Input and Debouncing	4
5.3 Saving Scores with EEPROM	4
5.4 Difficulty Scaling	4
5.5 Timer and Delay	5
5.6 Tone Generation	5
5.7 I2C and LCD	5
6. Game Modes	5
6.1 Solo Sprint	5
6.2 Sync Duel	6
6.3 The Challenger	6
7. System Features	6
7.1 Scrollable Start Menu	6
7.2 Audio Feedback	6
7.3 Visual Sequence Display	7
7.4 Persistent High Score Leaderboard	7
7.5 Per-Press Timeout	7
7.6 Idle Sleep Mode	7
7.7 Multi-File Code Structure	7
8. Simulation	8
9. Results and Testing	9
9.1 Unit-Level Tests	9
9.2 Integration and Gameplay Tests	10
9.3 Observed Limitations	11
10. Conclusion	12
11. Reference List	12

1. Introduction:

The report presents the complete process of creating a memory-based gaming console which operates on an Arduino Uno microcontroller. The project concept originates from the traditional Simon memory game in which players must repeat back the sequence of lights that they observe.

The original concept was developed into a complete system which includes three distinct game modes, two-player functionality, sound and light feedback, and permanent score storage that persists through device power shutdowns. The system uses an Arduino Uno as the main controller. The system connects to a 16x2 LCD screen and four LEDs and eight push buttons and a buzzer which all connect through a breadboard.

The code was written in Embedded C and divided into multiple files to create an organized structure that allowed for seamless collaborative development work. The report presents all project objectives to evaluate how the team integrated hardware and software components while testing their complete system development through various game modes.

2. Project Objectives:

The main goals we wanted to achieve with this project were:

- Build a working menu system that lets the player choose between three different game modes using the buttons.
- Implement all three game modes: Solo Sprint for a single player, Sync Duel for two players competing at the same time, and The Challenger where one player creates a sequence for the other to copy.
- Make sure button presses are detected reliably without accidentally registering twice. This was done using a software debounce check with a 30 millisecond delay.
- Use the four LEDs to display the sequence and the buzzer to play different sounds depending on what happens, so the player always knows if they got it right or wrong.
- Save the high scores for both players in the Arduino's EEPROM so they are not lost when the device is turned off.
- Make the game gradually harder as the player improves, by reducing how long each LED stays on during the sequence flash.
- Add a timeout so the game moves on if a player takes more than 5 seconds to press a button.
- Turn the display off automatically after 2 minutes of no activity to save power, and wake it back up when any button is pressed.
- Keep the source code split into separate files, with each file handling one part of the system, making it easier to understand and work on.

3. System Overview:

The system is controlled by an Arduino Uno microcontroller. When it is turned on, it loads any saved scores, shows a welcome screen, and then goes to the main menu. From the menu, the player picks a game mode using their buttons.

We designed the code around something called a state machine. This just means the program is always in one of a few defined states, like 'showing the menu' or 'playing Solo Sprint', and it switches between them depending on what the player does. This made the code much easier to manage because each state has its own function that handles everything for that part of the game.

The six states we used are:

- Menu – the home screen where the player chooses a mode.
- Solo Sprint – the single-player game.
- Sync Duel – both players play at the same time.
- The Challenger – Player 1 sets a sequence for Player 2 to copy.
- Score Display – shows the final scores after a game ends.
- Sleep – the screen turns off after 2 minutes of no activity to save power.

The main loop of the program runs continuously. Every time through the loop it checks all the buttons and then calls whichever function matches the current state. This happens so fast that the system feels instant and responsive to button presses.

4. Hardware:

4.1 Component List

Components List					
No.	Component Name	Description	Quantity	Unit Price (EGP)	Total Price (EGP)
1	Arduino uno	Microcontroller Board	1	467	467
2	Push buttons	Input buttons	8	0.75	6
3	LCD Display	16X2 display	1	145	145
4	Bread Board	Testing board	2	35	70
5	Jumper Wires	Connection Wires	2	35	70
6	Batteries	Primary power source	3	95	285
7	Battery Holder	3X Battery Holder	1	18	18
8	Resistors	220 ohm Resistors	10	0.25	2.5
9	LEDs	Indicator Lights	4	1.5	6
10	Small Buzzer	Audio Feedback Device	1	8	8
					1077.5

➤ Project Components and total cost estimation.

4.2 Hardware Implementation

All the components are connected on a breadboard using jumper wires. The Arduino receives its power supply from three 3.7V batteries.

The four LEDs on the Arduino board connect to digital pins 6, 7, 8, and 9. Each LED uses a 220Ω resistor to control its electrical current which flows through the system. The LEDs light up to show the sequence the player needs to copy, and they also flash briefly when the player presses a button to confirm the input.

Player 1 uses buttons connected to pins 2, 3, 4, and 5. Player 2 uses the analogue pins A0, A1, A2, and A3. We set all eight buttons to use the Arduino's built-in pull-up resistors, which means each button reads HIGH normally and LOW when pressed. The breadboard space remained clear because we used built-in pull-up resistors to handle button connections.

The LCD screen connects to the Arduino using only four wires, using an I2C module on the back of it. The two data wires (SDA and SCL) connect to pins A4 and A5. The screen displays all game menus together with instructions and scoring information. The device includes a backlight which we disable during sleep mode.

The buzzer connects to pin 10 of the system. The Arduino `tone()` function enables us to create various sounds which play through the system based on game events. The Arduino contains a small memory chip which functions as EEPROM storage. We use it to save the high scores for both players. It needs only 5 bytes of storage because the data maintains its existence even when the Arduino becomes unpowered..



Figure 1: Hardware implementation of the system showing components and wiring connections.

5. Software and Code:

We rewrote the entire codebase in embedded C after realising the requirement for the assignment was using embedded and not C, targeting the ATmega328P directly. Every Arduino library function has been replaced with register-level code. The firmware is split across many source files, each responsible for one hardware driver or software subsystem. This structure keeps the code organised, makes each module easy to test on its own, and reflects the kind of modular design used in professional embedded software development.

5.1 How the Game Software Works

The most important part of our code design is the state machine. We have a variable called `g.state` which always holds the name of the current state. In the main loop, a switch statement checks this variable and calls the correct function for whichever state is active. Each of those functions handles everything needed for that state. When the player does something that should change the state, the function just updates `g.state` and the main loop automatically starts calling the new function on the next iteration.

5.2 Button Input and Debouncing

We quickly noticed during testing that pressing a button once could sometimes register two or three times because of how the physical contacts work inside the button. This is a common problem called contact bounce. To fix it, we wrote a function called `input_update()` in `input.c` that reads all eight buttons directly from the PIND and PINC hardware registers. It only accepts a button press as a valid event if the button has been held in the same position for at least 30 milliseconds. This is long enough to filter out the bounce but short enough that the player does not notice any delay. Buttons are wired with `INPUT_PULLUP` so they read LOW when pressed and HIGH when released. The debounce logic tracks the last stable state and timestamp for all eight buttons independently.

5.3 Saving Scores with EEPROM

The ATmega328P has 1 KB of internal EEPROM. We access it directly through the hardware registers in `eeeprom_hal.c`, replacing the Arduino EEPROM library entirely. We use 5 bytes in total: 2 bytes for Player 1's high score, 2 bytes for Player 2's high score, and 1 byte as a magic flag at address 4. On a brand new chip the EEPROM contains random values, so without this check the game might treat those values as real scores. We check for a specific value (0xAB) and if it is not there we write both scores as zero and store the flag. We also use update functions that only write to EEPROM when the value has actually changed, which protects the chip's rated 100,000 write cycle limit.

5.4 Difficulty Scaling

To make the game harder as the player gets better, we reduce the time each LED stays on during the sequence flash. A new player gets 600 milliseconds per LED, which is easy enough to follow. Each time they score a point, that time goes down by 50 milliseconds. The minimum is 150 milliseconds, which is quite fast and challenging to keep up with. This calculation is done in `game_logic.c` using the `LED_ON_BASE_MS` and `DIFFICULTY_STEP` constants defined in `config.h`.

5.5 Timer and Delay

In the Arduino version we used Arduino's built-in `millis()` and `delay()`. In the embedded C version these are replaced by our own `timer.c` module. We configure Timer0 in CTC mode with a prescaler of 64 and a compare value of 249, which produces a compare match interrupt every exactly 1 millisecond at 16 MHz. The interrupt service routine increments a volatile 32-bit counter. Our `millis()` function reads this counter atomically by disabling interrupts briefly, and `delay_ms()` simply spins until the required number of milliseconds has passed.

5.6 Tone Generation

The Arduino version used `tone()` and `noTone()` to drive the buzzer. In our embedded C rewrite, `tone.c` controls Timer1 directly. Timer1 is a 16-bit timer configured in Fast PWM mode with the TOP value set by ICR1. The OC1B output on pin PB2 (D10) toggles on each compare match, producing a square wave at the target frequency. The ICR1 value is calculated from the desired frequency at runtime, allowing the buzzer to play any pitch needed for the different game sounds.

5.7 I2C and LCD

The Arduino version used the Wire library and LiquidCrystal_I2C library to communicate with the LCD screen. In our embedded C version, `i2c.c` drives the ATmega328P's hardware TWI peripheral directly using the TWCR, TWDR, TWSR, and TWBR registers. TWBR is set to 72 to produce a 100 kHz SCL clock at 16 MHz. On top of `i2c.c`, `lcd.c` implements the full HD44780 initialisation sequence in 4-bit mode via the PCF8574 I2C backpack, including functions for clearing the screen, setting the cursor position, printing text and numbers, and controlling the backlight.

6. Gameplay Modes:

6.1 Solo Sprint

This is the classic one-player memory challenge. Each round opens with the full sequence flashed on the LEDs, and it gets longer by one step every time the player succeeds. The player then must reproduce the whole thing from memory using their four buttons. They have five seconds between button presses. This creates a challenging and quick-thinking game, as one wrong press will end the game.

The LED flash speed starts at 600 ms per step and drops by 50 ms with every point scored, down to a minimum of 150 ms. By the time a player has earned nine points, they are watching LEDs that barely flash, which puts real pressure on visual recall. Getting to round ten or above on this mode requires concentration.

6.2 Sync Duel

Sync Duel is built for two players sitting next to each other. Both players watch the same sequence flash, then both enter it simultaneously — there is no turn order, no waiting. Each player's input is tracked completely independently, so one slow player does not affect the other at all. Once both have finished (or the 5-second timeout fires for a player who has not responded), the game compares results and hands out points accordingly.

There are four outcomes per round: both correct (one point each, win fanfare plays), only Player 1 correct (P1 gets the point), only Player 2 correct (P2 gets the point), or both wrong (no points, wrong tone). The design decision to reset the sequence length to zero after each round — rather than letting it grow as in Solo Sprint — was deliberate. It keeps each round self-contained and fast, which suits the competitive side-by-side format. The tension comes from reacting faster than the other person, not from managing an ever-growing sequence.

6.3 The Challenger

This mode is the most unusual of the three and the one that prompted the most back-and-forth in our design discussions. The idea is that Player 1 gets to choose the sequence that Player 2 has to beat — which immediately opens up strategy and bluffing. But to stop Player 1 from entering something they themselves cannot remember, we built in a proof phase. The mode runs through four stages

7. System Features

7.1 Scrollable Start Menu

The menu was one of the first things we got working, partly because it let us test button input before the game logic existed. Player 1's button 1 scrolls up through the three modes, button 2 scrolls down (it wraps around from the bottom back to the top), and button 3 launches whatever is highlighted. The LCD shows the selected mode on the top line with a '>' cursor, and the next mode in the list on the bottom line, giving a sense of the list without needing a third row. We added a menuDrawn flag so the display only refreshes when the cursor actually moves — otherwise the I2C bus was getting hammered with identical writes on every loop iteration, which caused visible flicker.

7.2 Audio Feedback

We put a lot of thought into making the audio distinctive enough that players do not have to glance at the LCD to understand what just happened. There are five different sounds and each has a clear character:

- Correct answer: 1000 Hz for 200 ms. A clean, bright beep that feels rewarding.
- Wrong answer: 300 Hz for 400 ms. Low and buzzy — noticeably unpleasant without being obnoxious.
- Game over: three descending tones (250 Hz, 150 Hz, 100 Hz, 200 ms each). The falling pitch makes it feel appropriately final.
- Menu selection: 700 Hz for 80 ms. Short and crisp, just enough to confirm a button press without becoming annoying during fast scrolling.

- Win fanfare: four ascending notes — C5, E5, G5, C6 — at 150 ms each. A recognisable mini-melody that feels like a small celebration.

7.3 Visual Sequence Display

The `flashSequence()` function handles all LED sequence playback. It walks through the sequence array and for each entry it turns the LED on for `ledOnMs()` milliseconds, then off, then waits 200 ms before the next one. Before the first flash there is a 400 ms pause with all LEDs off — we added that after early testing where the sequence started so abruptly that players were not ready. The function blocks during the flash because we deliberately do not want any button input to register while the sequence is playing. That brief blocking period is one of the few places in the code where we accepted a `delay()` rather than fighting to make it non-blocking.

7.4 Persistent High Score Leaderboard

Saving high scores was a non-negotiable requirement from the brief. Our implementation stores two 16-bit integers in EEPROM and checks a magic byte on boot to handle the first-run case cleanly. The scores only write to EEPROM when beaten, which protects the chip's write cycle budget. There is also a reset mechanism: holding all eight buttons for one second on the menu screen wipes both scores and plays the game-over sound to confirm it. We kept that gesture deliberately awkward — requiring all eight buttons at once — so nobody wipes the leaderboard by accident.

7.5 Per-Press Timeout

Every game mode has a 5-second timer on button input. The timer resets each time a valid button is pressed, so active players who are moving at a normal pace never feel it. It only kicks in if someone freezes completely — either because they have forgotten the sequence or have walked away from the game. In Solo Sprint a timeout ends the game; in Sync Duel and Challenger, it ends the current round. We track the timer using a `millis()` timestamp stored in `g.turn startMs`, which makes it completely compatible with the non-blocking loop structure.

7.6 Idle Sleep Mode

If nobody interacts with the device for two minutes, the system automatically enters the sleep state. The LCD backlight switches off, the screen is cleared, and all LEDs go dark. A brief wake prompt appears once. Any button press brings everything back. We added this mostly for practical reasons — if the device is left running in a lab or at a desk, there is no reason to keep the backlight burning. It also gives the project a slightly more professional feel, the kind of idle behaviour you would expect from any real consumer device.

7.7 Multi-File Code Structure

Splitting the code into nine files was something we agreed on at the start and stuck to throughout. It meant that whoever was working on, say, the audio functions did not need to understand or touch the EEPROM code, and vice versa. Forward declarations in `game.h` gave every module access to every function without circular dependencies. The structure also made incremental testing much easier — we could stub out an unfinished module and test the rest of the system around it.

8. Simulation:

We tested the system using a simulation to make sure everything works as expected before implementing it on hardware. The simulation helped us check how the system reacts to different inputs and whether the game logic is correct.

You can view the simulation here:

[\[Click here\]](#)

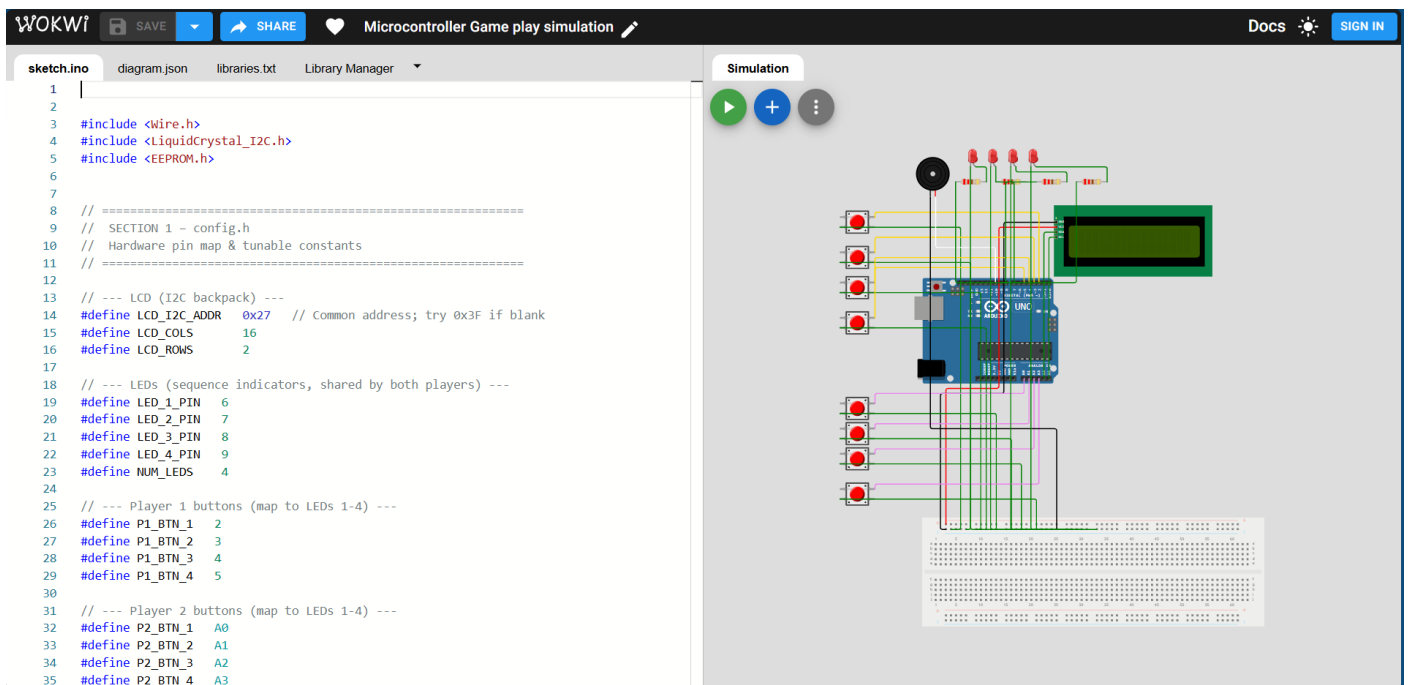


Figure 2:Simulation of the system during operation

9. Results and Testing

We tested in two stages. First, individual modules were checked in isolation to make sure the building blocks worked correctly before we connected them together. Then we ran the full system on real hardware with both players present to catch anything that only shows up in actual play — timing issues, confusing LCD messages, audio cues that did not land right. Simulation was not much use here because too many of the behaviours we needed to verify (debounce timing, EEPROM read/write, I2C LCD, buzzer frequencies) simply do not behave the same way in a software emulator as they do on the physical board.

9.1 Unit-Level Tests

Test Case	Outcome
Button debounce	Flicking a button faster than 30 ms consistently produced exactly one event per press. No phantom inputs appeared after tuning the DEBOUNCE_MS constant.
p1AllHeld() / p2AllHeld()	Holding all four buttons returned true; releasing any single button immediately returned false. The score-wipe combo triggered reliably at the one-second mark.
growSequence()	Calling the function 25 times in a row kept activeSeq.length capped at 20 (MAX_SEQ_LEN) with no out-of-bounds writes.
checkInput() – wrong input	Pressing a wrong button at positions 1, 3, and 5 of a known sequence all returned 0 immediately.
checkInput() – partial correct	Entering the first N-1 correct buttons of sequences of lengths 3, 7, and 15 consistently returned -1 on each intermediate press.
checkInput() – full correct	Completing a sequence of length 5 returned 1 only on the final (5th) press, not before.
ledOnMs() difficulty curve	Verified manually for scores 0–12: values were 600, 550, 500, 450, 400, 350, 300, 250, 200, 150, 150, 150, 150 — correct floor behaviour from score 9 onwards.
EEPROM persistence	Saving a score of 8, power-cycling, and rebooting showed 8 as the high score. Triggering the wipe combo and rebooting showed 0. A fresh board with uninitialised EEPROM defaulted cleanly to 0 on first boot.

9.2 Integration and Gameplay Tests

Test Case	Outcome
Solo Sprint – successful run	Completed rounds 1 through 6 in sequence. Score incremented correctly, sequence grew by one step each round, and LED flash speed noticeably shortened by round 5.
Solo Sprint – wrong button	Pressing an incorrect button on round 4 triggered audioWrong() and audioGameOver() immediately, displayed the correct error messages, and moved to STATE_SCORE_DISPLAY.
Solo Sprint – input timeout	Stopping all input after a correct partial entry triggered 'Time's up! / Game Over!' after exactly 5 seconds and ended the session.
Sync Duel – both correct	Both players entering the correct sequence within the window awarded one point each and played the win fanfare. Scores updated on the LCD correctly.
Sync Duel – split result	Player 1 correct and Player 2 wrong awarded only Player 1 a point, played audio Correct() (not audioWin()), and displayed 'P1 correct! / P1 +1 pt'.
Sync Duel – timeout	Neither player pressing anything within 5 seconds showed 'Timeout! / No points.' and reset the round without ending the game session.
Challenger – Player 2 success	Player 2, replicating the full sequence, correctly skipped Phase 3 entirely, awarded P2 the point, and showed CHALL_RESULT immediately.
Challenger – Player 2 fail, P1 proves	P2 entered a wrong button at step 2. P1 was then prompted and re-entered the full original sequence correctly, earning the point.
Challenger – proof failure	P1 set a sequence, P2 failed, then P1 also entered incorrectly during proof. 'P1 failed! / No points.' appeared and neither player scored.
Sleep mode activation	Leaving the device untouched on the menu for 120 seconds caused the LCD backlight to cut out and all LEDs to go dark. Pressing any button restored the menu instantly.
Score display and return to menu	After a Solo Sprint game over, STATE_SCORE_DISPLAY showed the correct score and declared the winner. Pressing any button returned to the menu with scores reset.
Secret score wipe	Holding all eight buttons for one second on the menu played audioGameOver(), showed 'Scores wiped!', and reloaded the menu with both high scores reset to zero.

9.3 Observed Limitations

Testing also turned up a few things we would change given more time:

- Sync Duel resets the sequence length after every round instead of letting it grow. We did this intentionally to keep rounds short, but it does mean the mode lacks the escalating-length pressure that makes Solo Sprint feel increasingly tense. Adding a configurable option to switch between reset-mode and grow-mode would improve it.
- In The Challenger, Player 2 only gets to see the sequence once. If they are briefly distracted during the flash — someone walks past, they blink at the wrong moment — there is no replay. A single retry option would make the mode feel fairer in casual settings.
- The I2C address for the LCD is a constant in config.h. If someone builds the project with a backpack that uses 0x3F instead of 0x27, they need to find and change that one line and recompile. It would be better to auto-detect or at least provide a clearer comment pointing to where the change needs to be made.
- The passive buzzer produces square waves, which can sound harsh at some frequencies. A small RC low-pass filter on the buzzer output, or switching to an active piezo element, would noticeably improve the audio quality without changing any firmware.



Figure 3: Final Hardware prototype showing the complete system with buttons, LEDs, and display interface

10. Conclusion

In conclusion, the project successfully delivered a fully functional microcontroller-based gaming system that meets the specified requirements. The system proved to be reliable, well-structured, and capable of handling real-world operation scenarios. This project highlights the effective integration of embedded C programming with hardware design.

11. Reference List:

[1] Microchip Technology Inc., 2018. ATmega328P Datasheet.

[2] Mazidi, M.A., Naimi, S. and Naimi, S., 2011. The AVR Microcontroller and Embedded Systems: Using Assembly and C. Pearson.

[3] Arduino, 2023. Arduino Documentation. Available at: <https://www.arduino.cc>